Sistema FCT

PHP 8.2 | Vue 3 | Vite 6 | MariaDB 10.4 | Docker Compose | ClamAV INSTREAM | Pinia state

Guía de incorporación y operaciones de desarrollo para el Sistema FCT, una aplicación web híbrida que gestiona prácticas de Formación en Centros de Trabajo (FCT).

> Docker Compose es el entorno de desarrollo recomendado. Esta configuración no es un despliegue de producción: los puertos y secretos de ejemplo son locales y deben cambiarse antes de compartir el entorno.

Inicio rápido

Con Git y Docker Desktop instalados, cuatro comandos dejan la aplicación funcionando en http://localhost:8080:

```
git clone https://github.com/campusedx/Practicas_Desarrollo.git
Set-Location .\Practicas_Desarrollo
Copy-Item .env.example .env
docker compose up -d --build
```

Después, opcionalmente, crea cuentas demo (ver sección 5) y consulta el estado de los servicios con `docker compose ps`.

Índice

1. Tecnologías
2. Requisitos previos
3. Preparar el entorno
4. Iniciar y detener la infraestructura
5. Base de datos, cuentas de desarrollo y phpMyAdmin
6. Recorrido funcional
7. Acceso a la aplicación y herramientas
8. Frontend y Vite HMR
9. Portal de documentación VitePress
10. Verificaciones y pruebas
11. Solución de problemas frecuentes

1. Tecnologías

- **Backend:** PHP 8.2, Apache y arquitectura MVC personalizada.

- **Frontend:** Vue 3, TypeScript, Pinia y Vite como islas progresivas sobre vistas HTML renderizadas por PHP.
- **Persistencia:** MariaDB 10.4 mediante PDO y consultas preparadas.
- **Seguridad de archivos:** ClamAV (TCP INSTREAM); los documentos se guardan en almacenamiento privado, fuera de `public/`.
- **Infraestructura:** Docker Compose v2 con volúmenes persistentes para la base de datos, almacenamiento y firmas antivirus.
- **Documentación:** portal VitePress con diagramas Mermaid.

2. Requisitos previos

- Git.
- Docker Desktop actualizado con Docker Compose v2 (`docker compose`, no el ejecutable antiguo `docker-compose`).
- Node.js 22–24 y npm para ejecutar Vite, pruebas de tipos o VitePress directamente en el host.
- PowerShell 5.1 o posterior para los comandos de esta guía en Windows.

El contenedor `app` incluye PHP 8.2, Apache y las extensiones PDO MySQL y mbstring; no es necesario instalar PHP en Windows para el flujo Docker.

La arquitectura se organiza en tres capas: presentación (HTML/PHP y pequeñas islas Vue), aplicación (rutas, middleware, controladores y reglas PHP) y datos (MariaDB y almacenamiento privado). Vue no sustituye la navegación MVC ni la autorización del servidor.

3. Preparar el entorno

Clona el repositorio:

```
git clone https://github.com/campusedx/Practicas_Desarrollo.git
```

Entra en la carpeta recién creada:

```
Set-Location .\Practicas_Desarrollo
```

Crea `.env` con los valores locales de referencia; Compose lo carga automáticamente desde la raíz:

```
Copy-Item .env.example .env
```

Abre `.env` para cambiar puertos o contraseñas si entran en conflicto con otros servicios:

```
notepad .env
```

Variables incluidas en la plantilla:

Variable	Predeterminado	Uso
`APP_PORT`	`8080`	Puerto HTTP de PHP/Apache en el host.
`APP_ENV`	`development` en `.env.example`; `production` si Compose no recibe un valor	Entorno de ejecución; el bypass de ClamAV sólo se permite en `development`, `test` o `testing`.
`FCT_CLAMAV_ENABLED`	`false` en las plantillas locales; `true` si Compose no recibe un valor	`false` omite el escaneo sólo con `APP_ENV=development`, `test` o `testing`; usa `true` para escanear.
`PMA_PORT`	`8082`	Puerto local de phpMyAdmin; se publica sólo en `127.0.0.1`.
`FCT_DB_NAME`	`sistema_fct`	Base de datos de desarrollo.
`FCT_DB_USER`	`fct`	Usuario que utiliza la aplicación y el formulario de acceso de phpMyAdmin.
`FCT_DB_PASS`	`fct_local_dev`	Contraseña del usuario de aplicación MariaDB.
`MARIADB_ROOT_PASSWORD`	`root_local_dev`	Contraseña administrativa de MariaDB, reservada para tareas locales de administración.
`FCT_VITE_DEV_SERVER`	vacío	URL pública del servidor Vite que PHP debe incluir durante desarrollo, por ejemplo `http://localhost:5173`.
`VITE_PORT`	`5173`	Puerto de Vite cuando se ejecuta desde el perfil Docker `frontend-dev`.

Los valores de `.env.example` son exclusivamente para desarrollo local; sustitúyelos por claves propias y no compartas ni confirmes `.env`.

Compose crea automáticamente el volumen privado `fct_storage` y lo monta como `/var/lib/fct/storage`, que es el valor de `FCT_STORAGE_PATH` dentro del contenedor. El Dockerfile prepara el directorio y sus permisos; no se debe mover a `public/`.

Si ejecutas PHP directamente con XAMPP en vez de Docker, configura `FCT_STORAGE_PATH` como una ruta absoluta a un directorio existente, con permisos de escritura para Apache y separado del document root. La aplicación rechaza rutas inexistentes o dentro de `public/`.

4. Iniciar y detener la infraestructura

Construye la imagen de la aplicación e inicia Apache/PHP, MariaDB, ClamAV y phpMyAdmin en segundo plano:

```
docker compose up -d --build
```

Comprueba que los servicios estén activos y revisa su estado de salud:

```
docker compose ps
```

ClamAV puede tardar varios minutos en descargar sus primeras firmas. El contenedor `app` espera a que MariaDB esté saludable, pero puede iniciar mientras ClamAV prepara sus firmas; si el análisis está habilitado, una carga se rechazará hasta que el antivirus responda correctamente. Consulta su salida si todavía está inicializando:

```
docker compose logs --tail=50 clamav
```

Comprueba que el almacenamiento privado exista y sea escribible por PHP:

```
docker compose exec app sh -c 'test -d "$FCT_STORAGE_PATH" && test -w "$FCT_STORAGE_PATH"'
```

Abre la aplicación en http://localhost:8080. Para detener los contenedores sin borrar datos:

```
docker compose down
```

Los volúmenes de base de datos, archivos subidos, firmas ClamAV y dependencias Docker persisten al detener los contenedores.

5. Base de datos, cuentas de desarrollo y phpMyAdmin

En la primera inicialización de un volumen MariaDB vacío, Compose importa `database/sistema_fct.sql` y aplica las migraciones de dominio, disponibilidad y ofertas, además de las tablas de webhooks, tokens API y sus respuestas. Los nombres de algunas migraciones comparten prefijo porque proceden de líneas de trabajo paralelas; Compose les asigna nombres de inicialización únicos para conservar el orden y evitar colisiones. La inicialización oficial de MariaDB sólo procesa estos archivos al crear una base de datos nueva; no vuelve a ejecutarlos en cada arranque.

Las ofertas históricas aprobadas no tienen horas exactas inferibles a partir de sus antiguos turnos amplios; se excluyen del matching y del catálogo hasta republicarlas con horario semanal. No asignes horarios aproximados automáticamente.

Para una base ya inicializada, no vuelvas a importar el esquema ni ejecutes migraciones existentes indiscriminadamente: varias no son idempotentes. Examina `docker compose ps` y los registros de MariaDB antes de diagnosticar una instalación incompleta. `009_desarrollo_aplicaciones_multiplataforma.sql` es idempotente y puede aplicarse a una base existente para añadir DAM. Para un entorno nuevo creado fuera de Compose, ejecuta el esquema y las migraciones una sola vez, en orden, contra esa base vacía.

Procedimiento para esquema y migraciones manuales. No canalices los archivos con `Get-Content -Raw … | docker compose exec …`: en Windows PowerShell 5.1, `$OutputEncoding` es US-ASCII y `Get-Content` lee como ANSI un UTF-8 sin BOM, de modo que los caracteres no ASCII llegan corruptos. Por ejemplo, `ENUM('Presencial','Remoto','Híbrido')` de `010_detalles_ofertas.sql` llegaría como `H??brido`. En su lugar, copia el archivo al contenedor `db` y ejecútalo desde allí:

```
docker compose cp (Resolve-Path .\database\sistema_fct.sql).Path db:/tmp/migracion.sql
'mariadb --user="$MARIADB_USER" --password="$MARIADB_PASSWORD" "$MARIADB_DATABASE" < /tmp/migracion.sql #' | docker
        compose exec -T db sh -s
```

Las comillas simples impiden que PowerShell expanda las variables: las resuelve `sh` dentro del contenedor. El `#` final convierte en comentario el `\r` que PowerShell 5.1 añade a la entrada estándar; sin él, ese `\r` pasa a formar parte de la ruta y `sh` responde `cannot open /tmp/migracion.sql`. `docker compose cp` escribe su progreso por stderr, y PowerShell 5.1 puede mostrarlo como `NativeCommandError`: decide el resultado con `$LASTEXITCODE`, no con ese texto. Este bloque carga el esquema inicial en la base seleccionada por las variables de Compose; úsalo sólo si el esquema aún no existe. Para aplicar todas las migraciones SQL en orden alfabético sobre esa base vacía:

```
foreach ($migration in Get-ChildItem .\database\migrations\*.sql | Sort-Object Name) {
    docker compose cp $migration.FullName db:/tmp/migracion.sql
    'mariadb --user="$MARIADB_USER" --password="$MARIADB_PASSWORD" "$MARIADB_DATABASE" < /tmp/migracion.sql #' |
        docker compose exec -T db sh -s
    if ($LASTEXITCODE -ne 0) { throw "Falló la migración $($migration.Name)" }
}
```

Cada migración se aplica una sola vez y en orden; no ejecutes este bucle sobre un volumen que ya haya recibido esas migraciones durante la inicialización de Compose. En particular, `009_webhook_responses.sql` no es idempotente (es un `ALTER TABLE` que falla si las columnas ya existen).

Para una base existente que ya tiene las migraciones de dominio hasta la 006, aplica `007_horarios_ofertas.sql` y `008_disponibilidad_indiferente.sql` en orden, y sólo si no se aplicaron previamente:

```
docker compose cp (Resolve-Path .\database\migrations\007_horarios_ofertas.sql).Path db:/tmp/migracion.sql
'mariadb --user="$MARIADB_USER" --password="$MARIADB_PASSWORD" "$MARIADB_DATABASE" < /tmp/migracion.sql #' | docker
        compose exec -T db sh -s
```

Este procedimiento crea la tabla de horarios exactos de las ofertas sin volver a ejecutar migraciones antiguas; reinicia el servicio `app` después de aplicarlo para que su comprobación de salud detecte la tabla.

Para una base que ya recibió `007_horarios_ofertas.sql`, aplica únicamente el campo de disponibilidad indiferente:

```
docker compose cp (Resolve-Path .\database\migrations\008_disponibilidad_indiferente.sql).Path db:/tmp/migracion.sql
'mariadb --user="$MARIADB_USER" --password="$MARIADB_PASSWORD" "$MARIADB_DATABASE" < /tmp/migracion.sql #' | docker
        compose exec -T db sh -s
```

`008_disponibilidad_indiferente.sql` añade `es_indiferente` de forma idempotente y migra el valor anterior de `turno` sólo la primera vez; `009_desarrollo_aplicaciones_multiplataforma.sql` inserta DAM si aún no existe. En una base que ya recibió la primera, aplica la segunda para habilitar DAM:

```
docker compose cp (Resolve-Path .\database\migrations\009_desarrollo_aplicaciones_multiplataforma.sql).Path
        db:/tmp/migracion.sql
'mariadb --user="$MARIADB_USER" --password="$MARIADB_PASSWORD" "$MARIADB_DATABASE" < /tmp/migracion.sql #' | docker
        compose exec -T db sh -s
```

El matching omite la cobertura horaria diaria para el alumnado indiferente, pero sigue exigiendo fechas, ciclo, oferta aprobada con franjas y capacidad.

`010_detalles_ofertas.sql` añade título, descripción, turno, modalidad, ubicación, requisitos y tecnologías a las ofertas existentes. En una base que ya recibió `009_desarrollo_aplicaciones_multiplataforma.sql`, aplícala una vez:

```
docker compose cp (Resolve-Path .\database\migrations\010_detalles_ofertas.sql).Path db:/tmp/migracion.sql
'mariadb --user="$MARIADB_USER" --password="$MARIADB_PASSWORD" "$MARIADB_DATABASE" < /tmp/migracion.sql #' | docker
        compose exec -T db sh -s
```

La migración es idempotente. Los datos de ofertas históricas reciben valores predeterminados compatibles; una descripción/ubicación no se inventa para esos registros. Después de aplicarla, comprueba con una consulta de solo lectura que `modalidad` conserva `Híbrido` (la columna debe mostrar `enum('Presencial','Remoto','Híbrido')`):

```
'mariadb --user="$MARIADB_USER" --password="$MARIADB_PASSWORD" "$MARIADB_DATABASE" -e "SHOW COLUMNS FROM
        empresa_especialidades_plazas LIKE ''modalidad''" #' | docker compose exec -T db sh -s
```

Las migraciones `007_webhooks.sql`, `008_api_access_tokens.sql` y `009_webhook_responses.sql` también se aplican al crear un volumen nuevo; Compose las ejecuta con nombres de inicialización únicos después de las migraciones de dominio. Si conservas una base creada antes de la integración de webhooks, aplica esas tres migraciones una sola vez, en ese orden y con el procedimiento `docker compose cp` descrito arriba, antes de desplegar el nuevo código. La última es un `ALTER TABLE` y no debe ejecutarse de nuevo si ya existen las columnas de respuesta.

Genera las cinco cuentas de prueba actuales (Administrador, Coordinador, Observador, Empresa y Alumno) sólo en la base local autorizada:

```
docker compose exec -e FCT_ENABLE_DEMO_SEED=1 -e FCT_DEMO_SEED_ALLOW_DOCKER=1 app php scripts/crear_cuentas_demo.php
```

El script imprime contraseñas aleatorias una sola vez. **Cada ejecución renueva todas las contraseñas demo y deja inválidas las anteriores**; guarda la salida de forma segura. También crea para el alumno demo disponibilidad de lunes a viernes, de 09:00 a 17:00, con franjas exactas. No ejecutes este comando en producción.
`database/seeds/001_sample_data.sql` es un conjunto legado de datos de muestra y no se ejecuta automáticamente ni sustituye al generador seguro de cuentas.

Consulta la matriz de roles y permisos para conocer el acceso restringido al catálogo y el alcance de solo lectura del perfil Observador.

Abre phpMyAdmin en http://localhost:8082. Inicia sesión con el servidor `db`, puerto `3306`, base `sistema_fct` y las credenciales `FCT_DB_USER` / `FCT_DB_PASS` de `.env` (por defecto, `fct`). El usuario `root` y `MARIADB_ROOT_PASSWORD` deben reservarse para administración local cuando sea necesaria; no uses phpMyAdmin para exponer el servicio a la red.

El puerto MariaDB publicado para clientes del host es `127.0.0.1:3307`; dentro de Compose, la aplicación y phpMyAdmin se conectan al nombre de servicio `db:3306`.

6. Recorrido funcional: disponibilidad, matching y seguimiento

El sistema compara ofertas y expedientes del mismo ciclo formativo, por ejemplo DAW con DAW y DAM con DAM. Para que una asignación sea válida, la oferta debe estar aprobada, tener franjas semanales publicadas y plazas disponibles; el periodo de disponibilidad del alumno debe cubrir el periodo completo de la oferta. El servidor vuelve a validar todos esos criterios al asignar, aunque la pantalla ya haya mostrado candidatos.

Los ciclos soportados incluyen **DAW** (Desarrollo de Aplicaciones Web) y **DAM** (Desarrollo de Aplicaciones Multiplataforma). La especialidad se relaciona por ID en la base de datos, y `009_desarrollo_aplicaciones_multiplataforma.sql` incorpora DAM sin duplicarlo. Una oferta guarda un único ciclo para preservar el matching actual.

La publicación recoge título, descripción, plazas, fechas, turno, franjas semanales de cinco minutos, modalidad, ubicación, requisitos y tecnologías. La modalidad presencial e híbrida requiere ubicación. La revisión y los listados muestran los detalles persistidos; el matching sigue usando la misma relación por ciclo, fechas y horarios.

En el portal del alumno, la disponibilidad tiene tres pasos de configuración:

1. Indica las fechas de inicio y fin del periodo.
2. Elige una de estas opciones horarias:
    - **Franjas exactas:** marca cada día y define su hora de inicio y fin. Puedes aplicar un horario general a varios días y luego editar cada día.
    - **Disponibilidad indiferente:** indica que aceptas cualquier horario de oferta; las franjas por día quedan desactivadas. Fechas y ciclo siguen siendo obligatorios.

3. Envía el formulario para guardar. La isla Vue muestra de forma reactiva el estado de guardado o el error; no guarda cambios automáticamente. Sin JavaScript, el formulario HTML/PHP mantiene el envío y la validación del servidor.

Con franjas exactas, cada intervalo ofertado `[inicio, fin]` debe quedar contenido en un intervalo de disponibilidad del mismo día (`disponibilidad.inicio <= oferta.inicio` y `disponibilidad.fin >= oferta.fin`). La opción indiferente omite únicamente esa comparación horaria: no omite el ciclo, las fechas, la aprobación, las franjas publicadas ni el control de plazas. Las ofertas históricas sin horarios semanales exactos no participan en el matching hasta volver a publicarse con su horario.

Administrador y Coordinador registran calificaciones de prácticas cerradas; se admiten notas de **0.00 a 10.00**, con hasta dos decimales y sin caracteres alfanuméricos. El alumno consulta sus propias notas. El tema claro/oscuro se administra mediante Pinia, conserva la elección en `localStorage` y ofrece un selector visual SVG. El perfil Observador es de solo lectura y recibe datos anonimizados, sin acceso a documentos privados. La política completa está en Roles y permisos.

Rol	Permisos principales
Administrador	Gestión de usuarios, ofertas/convenios y decisiones de revisión; asignaciones y calificaciones.
Coordinador	Seguimiento, asignaciones, cierre de prácticas, calificaciones y consultas; no aprueba ofertas o convenios mediante las rutas MVC.
Observador	Sólo lectura, con datos anonimizados y sin documentos privados; `POST /logout` sólo invalida su sesión.
Alumno	Su perfil, disponibilidad, asignaciones y calificaciones, más información de su empresa asignada.
Empresa	Sus convenios, sedes y ofertas; no decide sus revisiones.
Invitado	Inicio, registro, autenticación y recuperación; las áreas protegidas requieren sesión.

El botón de tema claro/oscuro sigue la preferencia del sistema si todavía no hay una elección guardada; al cambiarlo manualmente, la elección queda en el almacenamiento local del navegador.

La matriz anterior describe la navegación MVC protegida por `routes/web.php` . `public/api.php` es una API tokenizada separada y aplica su propia matriz; en la implementación actual, las revisiones de ofertas y convenios también están reservadas a Administrador. Comparte el sobre JSON de éxito/error de `JsonResponse` : éxito `{ "success": true, "data": T }` y error `{ "success": false, "error": "..." }` . En las listas, la paginación va en `data.items` y `data.meta` ; los códigos de error se envían en `X-API-Error-Code` . Consulta API REST antes de emitir tokens o integrar clientes; las vistas anonimizadas de Observador del MVC no equivalen a la salida REST.

Antivirus en desarrollo local

Compose ejecuta ClamAV y PHP se conecta a él por la red interna del proyecto. Si no se definen variables de entorno, Compose usa `APP_ENV=production` y `FCT_CLAMAV_ENABLED=true` ; las plantillas locales pueden reemplazarlos explícitamente. Para pruebas locales sin antivirus, define ambos `APP_ENV=development` y `FCT_CLAMAV_ENABLED=false` ; el modo de bypass registra una advertencia y también se admite en los entornos `test` y `testing` . Para probar el escaneo real, establece `FCT_CLAMAV_ENABLED=true` y espera a que `docker compose ps` muestre `clamav` saludable. En producción, deja el escaneo habilitado: configuración inválida, fallo de conexión, firmas caducadas, timeout o detección de amenaza rechazan la carga.

Si necesitas comprobar el comportamiento de fallo cerrado, establece `FCT_CLAMAV_ENABLED=true` , detén ClamAV sólo en un entorno local desechable (`docker compose stop clamav`), confirma que una carga se rechaza y vuelve a iniciarlo (`docker compose up -d clamav`). No desactives el análisis fuera de desarrollo ni expongas el puerto interno del scanner.

7. Acceso a la aplicación y herramientas

Servicio	Dirección local
Aplicación PHP	http://localhost:8080 (o `APP_PORT`)
phpMyAdmin	http://localhost:8082 (o `PMA_PORT`)
Vista previa de disponibilidad (sólo desarrollo)	http://localhost:8080/index.php?action=preview-disponibilidad
Vista previa de publicar oferta (sólo desarrollo)	`http://localhost:8080/dev/preview-publicar-oferta`
Vite HMR	http://localhost:5173 cuando se inicia el frontend
Portal VitePress	http://localhost:5174 con el comando de desarrollo de documentación

El acceso a páginas y acciones dentro de la aplicación continúa gobernado por `routes/web.php` y el rol de la sesión; conocer la URL no concede permisos.

8. Frontend y Vite HMR

Instala las dependencias fijadas en `package-lock.json` para trabajar con Vite y Vue desde el host:

```
npm ci
```

Define la URL del servidor Vite en `.env` para que las vistas PHP incluyan el cliente HMR y los módulos frontend durante desarrollo:

```
FCT_VITE_DEV_SERVER=http://localhost:5173
```

Aplica el cambio de entorno recreando el servicio PHP si ya estaba iniciado:

```
docker compose up -d app
```

Inicia Vite en el puerto usado en la URL anterior:

```
npm run dev
```

Vite sirve HMR en http://localhost:5173. Si el puerto está ocupado, libera el puerto o cambia `VITE_PORT` (perfil Docker) y `FCT_VITE_DEV_SERVER` para que la URL configurada coincida con la URL accesible desde el navegador.

También puedes iniciar el servidor Vite en un contenedor con el perfil opcional:

```
docker compose --profile frontend-dev up -d
```

El perfil instala dependencias con `npm ci`, ejecuta Vite escuchando en todas las interfaces del contenedor y publica `VITE_PORT` (5173 por defecto). Para regresar al bundle de producción incluido en la imagen, detén el perfil y vacía `FCT_VITE_DEV_SERVER` en `.env`.

Las vistas siguen renderizándose en PHP y conservan sus formularios como fallback; Vue se monta sólo en elementos marcados `data-vue-island`. Las acciones JSON del MVC usan el contrato `{ success, data }` / `{ success, error }`, envían `X-CSRF-Token` y no reemplazan la navegación MVC por rutas SPA. La API tokenizada de `public/api.php` conserva ese sobre discriminado y añade `items` / `meta` en las listas; consulta API REST.

CSS compartido y PostCSS

`frontend/src/main.ts` importa la hoja activa `public/assets/css/style.css`, por lo que Vite aplica el PostCSS configurado en `postcss.config.js` (`postcss-nested` y Autoprefixer; `vite.config.ts` lo referencia en `css.postcss`) y publica el CSS junto al bundle de Vue. En la imagen Docker, la etapa `frontend-build` del `Dockerfile` copia `postcss.config.js` y `public/assets/css` (además de `tsconfig.json`, `vite.config.ts` y `frontend`) antes de ejecutar `npm run build`; sin esa configuración, Vite omite PostCSS sin error y el bundle se generaría sin reglas anidadas resueltas ni prefijos, por lo que conviene que ambos archivos sigan en esa etapa. El layout PHP carga el CSS compilado desde el manifiesto de Vite en producción. En desarrollo (`FCT_VITE_DEV_SERVER` definido) Vite inyecta el CSS mediante JavaScript, que se ejecuta de forma diferida; para evitar un parpadeo sin estilos al cambiar de pantalla, `app/Views/layouts/main.php` enlaza además `assets/css/style.css` de forma bloqueante en el `<head>`. Si no hay bundle ni servidor HMR, ese mismo enlace actúa como fallback. Vite ya minifica los assets de producción; no se añade Tailwind ni CSSnano porque el proyecto no usa esas herramientas.

Verifica ambos lados del pipeline después de tocar los estilos:

```
npm run typecheck
npm run build
npm run docs:build
```

Webhooks para integraciones externas

La integración actual usa un destino saliente único configurable por entorno, no una tabla de endpoints registrados por usuario. Configura en `.env` `FCT_WEBHOOK_OUTBOUND_URL` y `FCT_WEBHOOK_OUTBOUND_SECRET`; el secreto de entrada se configura por separado con `FCT_WEBHOOK_INBOUND_SECRET`. No guardes secretos reales en Git. Compose pasa esas variables al contenedor PHP.

Cuando `FCT_WEBHOOK_OUTBOUND_URL` está configurada, los eventos se guardan en `webhook_outbox` dentro de la misma transacción que la operación de negocio y se envían después mediante el worker CLI; si no hay destino, no se encolan. Los eventos recibidos se registran en `webhook_inbox`. Se emiten `oferta.creada`, `oferta.revisada`, `convenio.creado`, `convenio.revisado`, `practica.asignada`, `practica.cerrada` y `student.availability_updated`. El evento de disponibilidad lleva `student_id`, `fecha_inicio`, `fecha_fin`, `turno`, `es_indiferente` y `horarios`; no incluye nombre ni correo.

El worker firma el cuerpo JSON exacto con HMAC-SHA256: `X-FCT-Timestamp` contiene la marca temporal y `X-FCT-Signature` lleva `sha256=` seguido de la firma de `timestamp.cuerpo`. El endpoint receptor verifica también la firma y la ventana temporal. Las entregas pendientes se reintentan mediante `scripts/dispatch_webhooks.php`; requiere PHP CLI con la extensión cURL habilitada, URL y secreto de salida válidos, y habilitar la ejecución explícitamente con `FCT_WEBHOOK_DISPATCH_ALLOW=1`:

```
docker compose exec -e FCT_WEBHOOK_DISPATCH_ALLOW=1 app php scripts/dispatch_webhooks.php
```

En la imagen Docker actual, PHP cURL está disponible: `docker compose exec -T app php -m` lista `curl` (la imagen base `php:8.2-apache` la incluye; el `Dockerfile` no la instala por separado). Si cambias la imagen base, repite `php -m` dentro de `app` antes de programar el worker; con un PHP CLI externo, comprueba que cURL esté habilitado. Esta comprobación confirma la extensión, pero no se ha verificado la entrega desde la imagen Docker actual. El límite opcional se configura con `FCT_WEBHOOK_DISPATCH_LIMIT` (1–100; por defecto 25). Configura el worker operativo para entregar los eventos de forma

asíncrona. El destino remoto debe usar HTTPS y resolver a direcciones públicas; el worker permite HTTP sólo para pruebas en loopback. Para registrar el receptor local y verificar el flujo completo, sigue la guía de webhooks y el script `scripts/local_webhooks.ps1`.

Cuando los cambios de una isla Vue no aparecen

La isla de disponibilidad actual se llama `AvailabilitySchedule` (archivo `frontend/src/islands/AvailabilitySchedule.vue`); el registro de `frontend/src/main.ts` descubre automáticamente los componentes Vue y la vista PHP debe usar el mismo valor en `data-vue-island`.

- **Desarrollo con HMR:** define `FCT_VITE_DEV_SERVER=http://localhost:5173` en `.env`, recrea PHP con `docker compose up -d app` para cargar esa variable, y ejecuta `npm run dev` en el host (o inicia el perfil `frontend-dev`). Comprueba que el navegador puede abrir `http://localhost:5173`.
- **Bundle de producción Docker:** `npm run build` genera `public/build` en el host, pero no actualiza una imagen PHP ya creada. Reconstruye y recrea la aplicación con `docker compose up -d --build app`; la imagen copia el bundle durante el build.
- **Comprueba el montaje:** abre la consola y la pestaña Network del navegador, fuerza recarga sin caché y confirma que se solicita el entrypoint de Vite o el bundle `build/assets/main-*.js`. Revisa que el HTML entregue el marcador `data-vue-island` y que su nombre corresponda a un archivo en `frontend/src/islands/`.
- **Si HMR conserva una versión antigua:** compara el módulo servido en `http://localhost:5173/frontend/src/islands/AvailabilitySchedule.vue` con el archivo del host. Si no coinciden, reinicia `docker compose restart frontend-dev` (o vuelve a iniciar el proceso `npm run dev`) para limpiar las transformaciones en memoria de Vite; después recarga el navegador sin caché. `docker compose down -v` no es necesario para invalidar HMR.
- **Aísla la vista:** `.env.example` establece `APP_ENV=development` para el entorno local; el valor de reserva de Compose es `production`. Las vistas previas (`preview-disponibilidad` y `preview-publicar-oferta`) sólo se registran cuando `APP_ENV` es `development` o `local` (`DevelopmentPreviewController::isEnabled()`). Usa `http://localhost:8080/index.php?action=preview-disponibilidad` para mostrar la isla con datos de prueba sin iniciar sesión; configura `APP_ENV=production` en despliegues para que las rutas no se registren. El bypass de ClamAV tiene un criterio distinto: sólo vale para `development`, `test` o `testing`, no para `local`.

Valida el frontend tras modificar componentes con `npm run typecheck` y `npm run build`. No hace falta purgar volúmenes para desplegar cambios de interfaz: basta con HMR o reconstruir la imagen de la aplicación.

9. Portal de documentación VitePress

Usa `npm ci` desde la raíz si aún no instalaste las dependencias Node. Para evitar competir con Vite HMR en el puerto 5173, ejecuta la documentación en el puerto 5174:

```
npm run docs:dev -- --port 5174
```

Abre http://localhost:5174; si el puerto indicado también está ocupado, VitePress asignará otro puerto disponible y lo mostrará en la terminal. El portal está en `docs/` y renderiza los diagramas Mermaid.

Compila el portal para verificar páginas Markdown, configuración y diagramas:

```
npm run docs:build
```

Sirve localmente la última compilación de VitePress:

```
npm run docs:preview -- --port 4173
```

10. Verificaciones y pruebas

Comprueba los tipos de los componentes e islas Vue/TypeScript:

```
npm run typecheck
```

Genera el bundle de producción y ejecuta de nuevo la comprobación de tipos:

```
npm run build
```

Ejecuta las pruebas de seguridad de almacenamiento y análisis de archivos en el contenedor PHP (las pruebas usan un directorio temporal y no requieren la base de datos):

```
$tests = (Resolve-Path .\tests).Path -replace '\\', '/'
docker compose run --rm --no-deps -v "${tests}:/var/www/html/tests:ro" app php tests/file_security.php
```

`tests/run.php` es una prueba de integración y debe apuntar exclusivamente a una base de datos desechable cuyo nombre empiece por `fct_test_`; configura `FCT_TEST_DB_NAME` y `FCT_DB_NAME` con el mismo nombre antes de ejecutarla. No la ejecutes contra `sistema_fct` ni una base que quieras conservar.

Restablecimiento de Base de Datos por Cambios de Esquema o Migraciones

> ⚠️ **ADVERTENCIA: restablecimiento destructivo.** `docker compose down -v` elimina todos los volúmenes nombrados de este proyecto Compose, no sólo el de MariaDB. Se perderán la base de datos (`db_data`), los archivos privados subidos (`fct_storage`), las firmas de ClamAV (`clamav_db`) y las dependencias Docker (`node_modules` y `frontend_node_modules`). Es irreversible; usa este procedimiento sólo en un entorno local desechable y después de guardar cualquier dato que quieras conservar.

Alternativa más segura si sólo necesitas reinicializar la base de datos: detén el proyecto, elimina únicamente el volumen `db_data` y vuelve a iniciarlo. Así se conservan el almacenamiento privado, ClamAV y las dependencias:

```
docker compose down
$dbVolume = (docker compose config --format json | ConvertFrom-Json).volumes.db_data.name
docker volume rm $dbVolume
docker compose up -d --build
```

Para restablecer **todos** los datos locales después de añadir cambios de esquema o migraciones, sigue estos tres pasos:

1. Elimina los contenedores y todos los volúmenes del proyecto:

   ```
   docker compose down -v
   ```

2. Reconstruye e inicia los servicios. MariaDB importará el esquema y ejecutará todas las migraciones configuradas al crear su volumen vacío, incluidas las tablas de webhooks:

```
docker compose up -d --build
```

3. Si necesitas las cuentas demo, créalas de nuevo en la base local:

```
docker compose exec -e FCT_ENABLE_DEMO_SEED=1 -e FCT_DEMO_SEED_ALLOW_DOCKER=1 app php
        scripts/crear_cuentas_demo.php
```

   El script renueva las contraseñas demo en cada ejecución; guarda la salida de forma segura. No lo ejecutes en producción.

11. Solución de problemas frecuentes

Puertos ocupados

Los puertos predeterminados son `8080` (aplicación), `8082` (phpMyAdmin), `3307` (MariaDB del host) y `5173` (Vite HMR). Cambia `APP_PORT`, `PMA_PORT` o `VITE_PORT` en `.env` para los puertos publicados configurables; el puerto de MariaDB se cambia editando el mapeo explícito `127.0.0.1:3307:3306` en `docker-compose.yml`. Si al abrir `http://localhost:8081` aparece Jenkins u otro servicio, eso indica que ese puerto ya está ocupado: establece `PMA_PORT=8082` (o cualquier puerto host libre) en `.env` y ejecuta `docker compose up -d` para publicar phpMyAdmin en el puerto elegido. Si VitePress comparte 5173 con Vite, usa el puerto 5174 recomendado.

Lista los puertos y estados publicados por Compose:

```
docker compose ps
```

Almacenamiento no escribible

Comprueba desde PHP que la ruta privada está montada y permite escritura:

```
docker compose exec app sh -c 'printf "FCT_STORAGE_PATH=%s\n" "$FCT_STORAGE_PATH"; test -d "$FCT_STORAGE_PATH" &&
        test -w "$FCT_STORAGE_PATH"'
```

En Docker, `FCT_STORAGE_PATH` debe seguir siendo `/var/lib/fct/storage`; si usas XAMPP, crea el directorio local fuera de `public/` y asigna permisos a la cuenta que ejecuta Apache.

MariaDB o ClamAV aún no saludables

Consulta el estado de los servicios:

```
docker compose ps
```

Consulta los últimos mensajes de MariaDB y ClamAV para encontrar errores de inicio o el progreso de descarga de firmas:

```
docker compose logs --tail=100 db clamav
```

El primer arranque de ClamAV puede tardar; espera a que indique `healthy` antes de investigar como fallo.

Para permitir cargas de prueba locales sin escaneo, configura explícitamente ambos valores: `APP_ENV=development` y `FCT_CLAMAV_ENABLED=false`; los registros de PHP indicarán que se omitió ClamAV. El bypass también se acepta en `APP_ENV=test` y `APP_ENV=testing`. Si se requiere análisis real, establece `FCT_CLAMAV_ENABLED=true` y revisa la conectividad interna a `clamav:3310`. Nunca uses el bypass en producción.

La página se ve sin estilos o sin islas Vue

Con `FCT_VITE_DEV_SERVER` definido, las islas y el CSS dependen de que Vite esté en marcha. Si el contenedor `frontend-dev` lleva mucho tiempo activo, puede servir una versión antigua de `frontend/src/main.ts`; reinícialo y recarga con Ctrl+F5:

```
docker compose restart frontend-dev
```

Comprueba que Vite responde con `curl.exe -s -o NUL -w "%{http_code}" http://localhost:5173/frontend/src/main.ts` (debe devolver `200`). Si no usas Vite, vacía `FCT_VITE_DEV_SERVER` en `.env` y recrea PHP con `docker compose up -d app`.

Ciclos DAW y DAM

Si no aparece DAM en una base ya existente, aplica una vez `database/migrations/009_desarrollo_aplicaciones_multiplataforma.sql` a esa base. En un volumen MariaDB nuevo, Compose ejecuta esa migración automáticamente. El matching compara IDs de especialidad: DAW sólo coincide con DAW y DAM con DAM.

Disponibilidad o controles de contraseña

Las horas se envían como `HH:MM` desde `<input type="time">` y el servidor acepta también `HH:MM:SS`; si una carga manual falla, revisa que días, indicador de disponibilidad indiferente y token CSRF correspondan al contrato. En formularios de contraseña, el botón de ojo cambia la visibilidad sin enviar el formulario; si no cambia, comprueba que `public/assets/js/password-toggle.js` esté cargado y que el campo siga siendo `type="password"` al cargar la página.

Reiniciar sin borrar datos

Reinicia los contenedores sin borrar volúmenes:

```
docker compose down
docker compose up -d --build
```

Para volver a inicializar la base de datos tras cambios de esquema o migraciones, consulta Restablecimiento de Base de Datos por Cambios de Esquema o Migraciones y revisa la advertencia sobre el alcance de `docker compose down -v`.

Más información

- Hoja de ruta de migración
- Flujo de datos y ciclo de vida
- Diagrama completo de arquitectura
- Configuración y fuentes de verdad: `routes/web.php`, `config/`, `database/` y `docker-compose.yml`.